

RETAILHUB

Airflow EC2 → SSM → Batch EC2 → Spark → S3 Gold
End-to-End Deployment & Troubleshooting Reference

Purpose
A practical future-reference document for the AWS Airflow portion of RetailHub: EC2 creation/configuration, Docker/Airflow setup, SSM orchestration, Silver and Gold DAGs, Spark environment changes, failures and fixes, validation commands, and private Airflow UI access.

Area	Current implementation
AWS Region	ap-south-1 (Mumbai)
Airflow EC2	RetailHub-Airflow-EC2 — private EC2, Amazon Linux 2023
Batch EC2	RetailHub-Batch-EC2 — private EC2, t3.small
Airflow	Apache Airflow 3.1.0 + LocalExecutor
Airflow DB	PostgreSQL 16 Docker container + named volume
Orchestration	Airflow → AWS Systems Manager (SSM) → Batch EC2
Spark	retailhub-spark-batch:4.2.0
Data	S3 Bronze → Silver → Gold
Gold output	s3://retailhub-gold-252bda/

Security note: This document intentionally uses placeholders for passwords/API keys. Do not paste secrets into documentation. The project currently uses a Batch .env for the RDS password because this is a demo deployment; Secrets Manager is the recommended production upgrade.

1. QUICK REFERENCE / TABLE OF CONTENTS

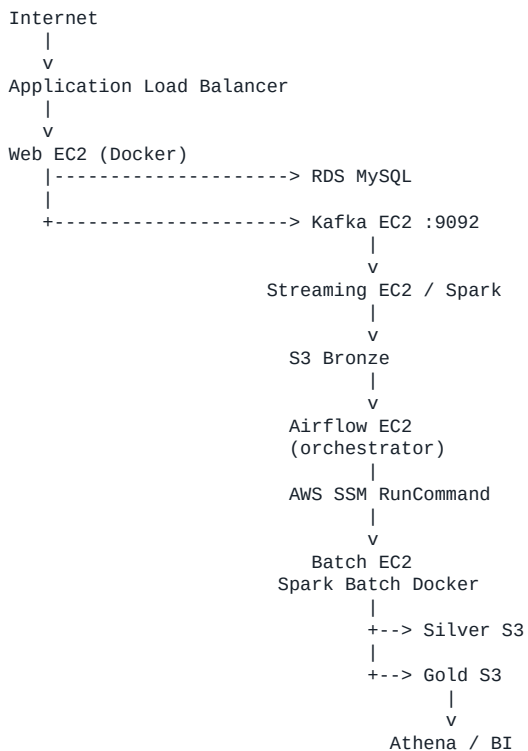
2. Architecture and data flow
3. Airflow EC2 creation and configuration
4. Docker installation and Airflow directory
5. Airflow Docker Compose configuration
6. Airflow startup, validation and UI
7. AWS IAM / SSM communication path
8. Batch EC2 and Spark Batch configuration
9. Silver DAG — full AWS code
10. Silver deployment and troubleshooting
11. Gold DAG — full AWS code
12. Gold deployment and validation
13. Problems encountered and exact fixes
14. Command reference / operational runbook
15. Current final state and next steps

Operating rule

Proceed one verification at a time. Do not trigger another DAG while the same DAG is already running unless intentionally testing concurrency. For Airflow, use docker compose down/up only after active Spark runs have completed.

2. ARCHITECTURE AND DATA FLOW

Target AWS flow



Important design change from local development: Airflow no longer starts the Spark Docker container locally through `/var/run/docker.sock`. Instead, Airflow uses its AWS IAM role and SSM to remotely execute Docker/Spark commands on Batch EC2.

Component	Location	Responsibility
Airflow EC2	Private subnet	Schedules and orchestrates Silver/Gold
Batch EC2	Private subnet	Runs Spark Batch Docker image
S3	AWS	Bronze, Silver, Gold, quarantine, checkpoints
RDS MySQL	Private	RetailHub relational database
SSM	AWS control plane	Secure command path Airflow → Batch
ALB	Public subnets	Public website entry point

3. AIRFLOW EC2 CREATION AND CONFIGURATION

The Airflow EC2 was created as a private compute node in the RetailHub VPC.

Setting	Value / choice
Name	RetailHub-Airflow-EC2
AMI	Amazon Linux 2023
Instance type	t3.small
Subnet	Private subnet
Public IPv4	Disabled / none
Security Group	RetailHub-Airflow-SG
IAM role	RetailHub-Airflow-Role
Inbound	No public Airflow port; UI accessed through SSM port forwarding
Outbound	Allowed for required AWS / Docker / package connectivity

Why private? Airflow is an internal orchestrator. Port 8080 is not exposed to the Internet. The browser reaches it through an authenticated SSM port-forwarding session.

SSM prerequisite: The Airflow role includes AmazonSSMManagedInstanceCore and the policies required to call SendCommand/GetCommandInvocation against Batch. The actual Airflow → Batch SSM command was tested successfully.

Verified SSM path

Airflow EC2 successfully executed AWS-RunShellScript on Batch EC2 and returned: RETAILHUB_SSM_TEST, Batch hostname, and Docker version 25.0.14.

Initial Docker installation / host setup

```
# Update packages
sudo dnf update -y

# Install Docker
sudo dnf install -y docker

# Start Docker now and on boot
sudo systemctl enable --now docker

# Verify
docker --version
sudo systemctl status docker

# Check Compose
docker compose version
```

4. DOCKER INSTALLATION, DIRECTORY AND AIRFLOW FILES

The Airflow EC2 working directory was created as:

```
/opt/retailhub-airflow/
├── config/
├── dags/
│   ├── silver_dag.py
│   └── gold_dag.py
├── plugins/
├── logs/
└── docker-compose.yml
```

Directory creation command used:

```
sudo mkdir -p /opt/retailhub-airflow/{config,dags,plugins,logs}
cd /opt/retailhub-airflow
```

The Airflow host logs directory required a permission correction because the Airflow container runs as UID 50000.

```
sudo chown -R 50000:0 logs
sudo chmod -R 775 logs
mkdir -p logs/dag_processor
sudo chown -R 50000:0 logs
sudo chmod -R 775 logs
```

Problem → fix

DAG processor initially failed with `PermissionError` on `/opt/airflow/logs/dag_processor`. Changing ownership to UID 50000 and group 0, then granting 775 permissions, fixed DAG processor logging.

Final AWS-adapted Docker Compose:

services:

```
airflow-db:
  image: postgres:16
  container_name: retailhub-airflow-db
  restart: unless-stopped
  environment:
    POSTGRES_USER: airflow
    POSTGRES_PASSWORD: airflow
    POSTGRES_DB: airflow
  volumes:
    - airflow-db-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "airflow"]
    interval: 10s
    timeout: 5s
    retries: 5

airflow:
  image: apache/airflow:3.1.0
  container_name: retailhub-airflow
  restart: unless-stopped
  depends_on:
    airflow-db:
      condition: service_healthy
  environment:
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@airflow-db:5432/airflow
    AIRFLOW__CORE__LOAD_EXAMPLES: "false"
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ./plugins:/opt/airflow/plugins
  ports:
    - "8080:8080"
  command: standalone
```

```
volumes:
  airflow-db-data:
```

Important: The AWS Compose deliberately does NOT mount `/var/run/docker.sock` and does not mount the local Spark data directory. That is because Spark now runs on Batch EC2 via SSM.

Start / stop commands:

```
cd /opt/retailhub-airflow
```

```
docker compose up -d
```

```
docker compose ps
```

```
docker compose logs -f airflow
```

```
# Safe restart after active jobs finish
```

```
docker compose down
```

```
docker compose up -d
```

```
# DO NOT use this for normal restart:
```

```
# docker compose down -v  
# because it removes the PostgreSQL metadata volume.
```

5. AIRFLOW STARTUP, VALIDATION AND UI

Core validation commands used during setup:

```
# Containers
docker compose ps

# Airflow health response
curl -I http://localhost:8080

# Airflow version
docker exec retailhub-airflow airflow version

# List DAGs
docker exec retailhub-airflow airflow dags list

# Find Silver
docker exec retailhub-airflow airflow dags list | grep retailhub_silver

# Find Gold
docker exec retailhub-airflow airflow dags list | grep retailhub_gold

# Check DAG report
docker exec retailhub-airflow airflow dags report

# Check scheduler jobs
docker exec retailhub-airflow airflow jobs check

# List DAG runs
docker exec retailhub-airflow airflow dags list-runs -d retailhub_silver
docker exec retailhub-airflow airflow dags list-runs -d retailhub_gold

# Task states for a specific run
docker exec retailhub-airflow airflow tasks states-for-dag-run retailhub_gold 2026-10-05T17:00:00+00:00
```

Airflow 3.1 CLI note

The project uses Airflow 3.1.0. The command form that worked for run history was `airflow dags list-runs -d retailhub_silver`. A previously attempted variant produced an unrecognized-argument error; use the working form above.

DAG pause state:

Airflow was configured with DAGs paused at creation. Therefore a newly loaded DAG can show **False** in the paused column after unpausing / enabling it, and it can then run on its schedule.

```
# Unpause Silver
docker exec retailhub-airflow airflow dags unpause retailhub_silver

# Unpause Gold
docker exec retailhub-airflow airflow dags unpause retailhub_gold
```

Manual single-task test:

```
# Example: run only dim_date for debugging/validation
docker exec retailhub-airflow airflow tasks test retailhub_gold dim_date 2026-10-05
```

Private Airflow UI via SSM

```
# Run on Windows PowerShell after installing AWS CLI
# and Session Manager Plugin.

aws ssm start-session `
  --target i-04287d3175fbb94d1 `
  --document-name AWS-StartPortForwardingSession `
  --parameters '{"portNumber":["8080"],"localPortNumber":["8080"]}' `
  --region ap-south-1

# Keep the PowerShell window open.
# Then browse to:
http://localhost:8080
```

Windows issue encountered

PowerShell first rejected the command because the instance ID was pasted as a Markdown hyperlink. The correct value is only the raw instance ID: `i-04287d3175fbb94d1`. The laptop then reported 'aws is not recognized', meaning AWS CLI was not installed/on PATH. Install AWS CLI and the Session Manager Plugin, open a new PowerShell, then retry.

6. AIRFLOW → SSM → BATCH EC2 COMMUNICATION

The key architectural change was replacing local Docker execution with SSM remote execution.

```

Airflow container
  |
  | boto3 / ssm.send_command()
  v
AWS Systems Manager
  |
  | AWS-RunShellScript
  v
Batch EC2
  |
  | docker run ...
  v
retailhub-spark-batch:4.2.0
  |
  v
Spark job → S3
  
```

Airflow Python calls:

```

ssm = boto3.client(
    "ssm",
    region_name="ap-south-1",
)

response = ssm.send_command(
    InstanceIds=["i-075044a1c7a02e50f"],
    DocumentName="AWS-RunShellScript",
    Parameters={"commands": [command]},
    Comment="RetailHub Spark Job",
)

command_id = response["Command"]["CommandId"]

result = ssm.get_command_invocation(
    CommandId=command_id,
    InstanceId="i-075044a1c7a02e50f",
)
  
```

IAM policies attached to the Airflow role included the SSM permissions needed for SendCommand and invocation status retrieval. A direct SSM test from Airflow to Batch proved the complete control path.

```

aws ssm send-command --region ap-south-1 --instance-ids i-075044a1c7a02e50f --document-name "AWS-RunShellScript" --parameters 'commands=
# Then inspect the returned CommandId:
aws ssm get-command-invocation --region ap-south-1 --command-id <COMMAND_ID> --instance-id i-075044a1c7a02e50f
  
```

Verified result

SSM returned Success and the Batch host returned its hostname plus Docker version 25.0.14. This proved Airflow EC2 → SSM → Batch EC2 → Docker.

7. BATCH EC2 AND SPARK BATCH CONFIGURATION

Setting	Value
Instance	RetailHub-Batch-EC2
Instance ID	i-075044a1c7a02e50f
AMI	Amazon Linux 2023
Type	t3.small
Network	Private subnet, no public IP
Security Group	RetailHub-Batch-SG
IAM Role	RetailHub-Batch-Role
Image	naveen9200/retailhub-spark-batch:4.2.0
Directory	/opt/retailhub-batch

Batch .env after the RDS connection was added:

```
BRONZE_PATH=s3a://retailhub-bronze-252bda/
SILVER_PATH=s3a://retailhub-silver-252bda/
GOLD_PATH=s3a://retailhub-gold-252bda/
QUARANTINE_PATH=s3a://retailhub-quarantine-252bda/events

MYSQL_HOST=retailhub-mysql.c3gee6cger7l.ap-south-1.rds.amazonaws.com
MYSQL_PORT=3306
MYSQL_DATABASE=retailhub
MYSQL_USER=retailhubadmin
MYSQL_PASSWORD=<RDS_PASSWORD>
```

The RDS variables are required by **dim_customer.py**. For this demo deployment the password was intentionally kept in the Batch .env, as chosen by the project owner. Production recommendation: AWS Secrets Manager.

Silver execution command proven on Batch:

```
docker run --rm --env-file /opt/retailhub-batch/.env -e BRONZE_PATH=s3a://retailhub-bronze-252bda -e SILVER_PATH=s3a://retailhub-silver-252bda
```

Gold execution pattern:

```
docker run --rm --env-file /opt/retailhub-batch/.env -e SILVER_PATH=s3a://retailhub-silver-252bda -e GOLD_PATH=s3a://retailhub-gold-252bda
```

Spark path correction

An earlier Gold version used `GOLD_PATH=s3://...`. The final DAG uses `s3a://retailhub-gold-252bda` so Spark's Hadoop/S3A connector writes correctly.

8. SILVER DAG — FULL AWS CODE

Final Silver DAG used in AWS:

```

from datetime import datetime, timedelta
import time

import boto3

from airflow import DAG
from airflow.exceptions import AirflowException
from airflow.providers.standard.operators.python import PythonOperator

AWS_REGION = "ap-south-1"
BATCH_INSTANCE_ID = "i-075044a1c7a02e50f"
SSM_DOCUMENT = "AWS-RunShellScript"

default_args = {
    "owner": "retailhub",
    "retries": 2,
    "retry_delay": timedelta(minutes=2),
}

def run_silver_on_batch():
    ssm = boto3.client("ssm", region_name=AWS_REGION)

    command = r"""
docker run --rm \
--env-file /opt/retailhub-batch/.env \
-e BRONZE_PATH=s3a://retailhub-bronze-252bda \
-e SILVER_PATH=s3a://retailhub-silver-252bda \
-e QUARANTINE_PATH=s3a://retailhub-quarantine-252bda/events \
naveen9200/retailhub-spark-batch:4.2.0 \
spark-submit \
/opt/retailhub/src/silver/bronze_to_silver.py
"""

    response = ssm.send_command(
        InstanceIds=[BATCH_INSTANCE_ID],
        DocumentName=SSM_DOCUMENT,
        Parameters={"commands": [command]},
        Comment="RetailHub Silver Spark Job",
    )

    command_id = response["Command"]["CommandId"]
    print(f"SSM Command ID: {command_id}")

    while True:
        try:
            result = ssm.get_command_invocation(
                CommandId=command_id,
                InstanceId=BATCH_INSTANCE_ID,
            )
            status = result["Status"]
            print(f"SSM Status: {status}")

            if status in [
                "Success", "Failed", "Cancelled",
                "TimedOut", "Undeliverable", "Terminated",
            ]:
                break

        except ssm.exceptions.InvocationDoesNotExist:
            print("SSM invocation not available yet. Waiting 5 seconds...")

            time.sleep(5)

        stdout = result.get("StandardOutputContent", "")
        stderr = result.get("StandardErrorContent", "")

        if stdout:
            print("===== SPARK OUTPUT =====")
            print(stdout)

        if stderr:
            print("===== SPARK ERROR =====")
            print(stderr)

        if status != "Success":
            raise AirflowException(
                f"RetailHub Silver Spark job failed. "
                f"SSM status: {status}. Error: {stderr}"
            )

    print("RetailHub Silver Spark job completed successfully.")

with DAG(
    dag_id="retailhub_silver",
    default_args=default_args,
    description="RetailHub Bronze to Silver pipeline",
    schedule="*/10 * * * *",
    start_date=datetime(2026, 9, 8),
    catchup=False,
    max_active_runs=1,
    tags=["retailhub", "silver", "aws", "ssm", "spark"],
) as dag:

    bronze_to_silver = PythonOperator(
        task_id="bronze_to_silver",
        python_callable=run_silver_on_batch,
    )

```

Silver schedule: every 10 minutes. It has one Spark task, **bronze_to_silver**.

Silver DAG behavior:

- Airflow creates an SSM command.
- Batch EC2 runs the Spark Batch Docker image.
- Spark reads S3 Bronze and writes S3 Silver.
- Airflow polls the SSM invocation until a terminal status.
- Spark stdout/stderr are printed into Airflow task logs.
- Any non-Success SSM status raises `AirflowException` and triggers Airflow retry.

9. SILVER DEPLOYMENT, VALIDATION AND TROUBLESHOOTING

Original Silver failure:

```
ssm.send_command(...) succeeded
CommandId returned:
deb01091-421a-46fe-8021-f09fd184d84e
```

```
Then immediate:
ssm.get_command_invocation(...)
```

```
Result:
botocore.errorfactory.InvocationDoesNotExist
```

Root cause: SSM had accepted the command, but the invocation record was not yet available when Airflow immediately polled it.

Fix:

```
while True:
    try:
        result = ssm.get_command_invocation(
            CommandId=command_id,
            InstanceId=BATCH_INSTANCE_ID,
        )
        status = result["Status"]

        if status in [
            "Success", "Failed", "Cancelled",
            "TimedOut", "Undeliverable", "Terminated",
        ]:
            break

    except ssm.exceptions.InvocationDoesNotExist:
        print("SSM invocation not available yet. Waiting 5 seconds...")

    time.sleep(5)
```

Result

After this fix, the manual Silver run completed with state **success**. The previously failing scheduled runs stopped failing because of the immediate invocation lookup race.

Silver validation:

```
# Trigger manually if required
docker exec retailhub-airflow airflow dags trigger retailhub_silver

# Inspect runs
docker exec retailhub-airflow airflow dags list-runs -d retailhub_silver

# Inspect a specific run
docker exec retailhub-airflow airflow tasks states-for-dag-run    retailhub_silver scheduled__2026-10-05T16:40:00+00:00

# Verify S3 Silver
aws s3 ls s3://retailhub-silver-252bda/ --recursive | tail -30
```

Operational caution: Do not trigger a second Silver run while a scheduled run is still running unless you intentionally want overlapping tests. The DAG has `max_active_runs=1`.

10. GOLD DAG — FULL AWS CODE

Final Gold DAG used in AWS:

```

from datetime import datetime, timedelta
import time

import boto3

from airflow import DAG
from airflow.exceptions import AirflowException
from airflow.providers.standard.operators.python import PythonOperator

AWS_REGION = "ap-south-1"

BATCH_INSTANCE_ID = "i-075044a1c7a02e50f"

SSM_DOCUMENT = "AWS-RunShellScript"

default_args = {
    "owner": "retailhub",
    "retries": 2,
    "retry_delay": timedelta(minutes=2),
}

def run_gold_on_batch(script):

    ssm = boto3.client(
        "ssm",
        region_name=AWS_REGION,
    )

    command = f"""
docker run --rm \
--env-file /opt/retailhub-batch/.env \
-e SILVER_PATH=s3a://retailhub-silver-252bda \
-e GOLD_PATH=s3a://retailhub-gold-252bda \
naveen9200/retailhub-spark-batch:4.2.0 \
spark-submit \
/opt/retailhub/src/gold/{script}
"""

    response = ssm.send_command(
        InstanceIds=[BATCH_INSTANCE_ID],
        DocumentName=SSM_DOCUMENT,
        Parameters={
            "commands": [command]
        },
        Comment=f"RetailHub Gold Spark Job - {script}",
    )

    command_id = response["Command"]["CommandId"]

    print("=" * 70)
    print("RETAILHUB GOLD JOB")
    print("=" * 70)
    print(f"Script      : {script}")
    print(f"SSM Command : {command_id}")
    print("=" * 70)

    while True:

        try:

            result = ssm.get_command_invocation(
                CommandId=command_id,
                InstanceId=BATCH_INSTANCE_ID,
            )

            status = result["Status"]

            print(f"SSM Status: {status}")

            if status in [
                "Success",
                "Failed",
                "Cancelled",
                "TimedOut",
                "Undeliverable",
                "Terminated",
            ]:
                break

        except ssm.exceptions.InvocationDoesNotExist:

            print(
                "SSM invocation not available yet. "
                "Waiting 5 seconds..."
            )

            time.sleep(5)

    stdout = result.get(
        "StandardOutputContent",
        "",
    )

    stderr = result.get(
        "StandardErrorContent",
        "",
    )

    if stdout:

        print("=" * 70)
        print("SPARK OUTPUT")
        print("=" * 70)

        print(stdout)

    if stderr:

        print("=" * 70)
        print("SPARK ERROR")
        print("=" * 70)

```

```

        print(stderr)

    if status != "Success":
        raise AirflowException(
            f"""
RetailHub Gold Spark job failed.

Script:
{script}

SSM Status:
{status}

Error:
{stderr}
"""
        )

    print("=" * 70)
    print(f"SUCCESS: {script}")
    print("=" * 70)

with DAG(
    dag_id="retailhub_gold",
    default_args=default_args,
    description="RetailHub Silver to Gold pipeline",
    schedule="*/15 * * * *",
    start_date=datetime(
        2026,
        9,
        8,
    ),
    catchup=False,
    max_active_runs=1,
    max_active_tasks=3,
    tags=[
        "retailhub",
        "gold",
        "aws",
        "ssm",
        "spark",
    ],
) as dag:
    fact_orders = PythonOperator(
        task_id="fact_orders",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "fact_orders.py",
        },
    )

    fact_order_items = PythonOperator(
        task_id="fact_order_items",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "fact_order_items.py",
        },
    )

    fact_payments = PythonOperator(
        task_id="fact_payments",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "fact_payments.py",
        },
    )

    fact_delivery = PythonOperator(
        task_id="fact_delivery",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "fact_delivery.py",
        },
    )

    fact_product_interactions = PythonOperator(
        task_id="fact_product_interactions",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "fact_product_interactions.py",
        },
    )

    dim_customer = PythonOperator(
        task_id="dim_customer",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "dim_customer.py",
        },
    )

    dim_product = PythonOperator(
        task_id="dim_product",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "dim_product.py",
        },
    )

    dim_category = PythonOperator(
        task_id="dim_category",
        python_callable=run_gold_on_batch,
        op_kwargs={
            "script": "dim_category.py",
        },
    )

```

```

dim_date = PythonOperator(
    task_id="dim_date",
    python_callable=run_gold_on_batch,
    op_kwargs={
        "script": "dim_date.py",
    },
)

```

Gold contains 9 independent PythonOperator tasks:

Type	Task ID	Spark script
Fact	fact_orders	fact_orders.py
Fact	fact_order_items	fact_order_items.py
Fact	fact_payments	fact_payments.py
Fact	fact_delivery	fact_delivery.py
Fact	fact_product_interactions	fact_product_interactions.py
Dimension	dim_customer	dim_customer.py
Dimension	dim_product	dim_product.py
Dimension	dim_category	dim_category.py
Dimension	dim_date	dim_date.py

Concurrency: max_active_tasks=3, so up to three Gold Spark jobs can run concurrently. max_active_runs=1 prevents overlapping Gold DAG runs.

11. GOLD DEPLOYMENT AND VALIDATION

Before full Gold execution, one task was deliberately tested: **dim_date**.

```
docker exec retailhub-airflow airflow tasks test    retailhub_gold dim_date 2026-10-05
```

Successful result:

```
=====
SUCCESS: dim_date.py
=====
```

Task instance state updated ... new_state=success

S3 verification confirmed:

```
aws s3 ls s3://retailhub-gold-252bda/ --recursive | tail -30
```

```
dim_date/_SUCCESS
dim_date/part-00000-e5f2...snappy.parquet
dim_date/part-00001-e5f2...snappy.parquet
```

Also present:
 dim_category/
 dim_customer/
 dim_product/
 fact_delivery/
 fact_order_items/
 fact_orders/
 fact_payments/

At the time of the final verification, the scheduled Gold run showed multiple tasks executing successfully/running:

Task	Observed state
fact_payments	success
fact_delivery	success
fact_product_interactions	running
fact_orders	running
fact_order_items	running
dim_customer	scheduled
dim_product	scheduled
dim_category	scheduled
dim_date	scheduled in that particular scheduled run

Inspect task states with:

```
docker exec retailhub-airflow airflow tasks states-for-dag-run    retailhub_gold <DAG_RUN_ID>
```

12. PROBLEMS ENCOUNTERED AND HOW THEY WERE FIXED

#	Problem	Cause	Fix / final state
1	Airflow DAG processor PermissionError	Container user could not write host-mounted logs	chown logs to UID 50000:0; chmod 775; create dag_processor directory
2	Silver DAG not visible / wrong filename	DAG file was initially named silver_day.py instead of silver_dag.py and host mount needed correction	Renamed/recreated file; force-recreated Airflow container; verified /opt/airflow/dags/silver_dag.py
3	SSM InvocationDoesNotExist	Airflow polled SSM immediately after send-command	Catch ssm.exceptions.InvocationDoesNotExist and wait 5 seconds
4	Airflow → Batch IAM concern	DescribeInstanceInformation was not available from role	Actual SendCommand/GetCommandInvocation path was tested; no unnecessary IAM introspection permission added
5	Gold used s3:// path	Spark expects S3A connector path in this configuration	Final Gold DAG uses s3a://retailhub-gold-252bda
6	dim_customer lacked DB credentials	Batch .env initially contained only S3 paths	Added MYSQL_HOST/PORT/DATABASE/USER/PASSWORD to Batch .env for demo
7	Airflow UI private	No public 8080 inbound by design	Use AWS-StartPortForwardingSession from Windows to localhost:8080
8	Windows aws command failed	AWS CLI not installed/on PATH	Install AWS CLI and Session Manager Plugin; reopen PowerShell
9	PowerShell target command failed	Markdown hyperlink was pasted instead of raw instance ID	Use only i-04287d3175fbb94d1
10	Spark image Python test failed	Image did not expose 'python' executable on PATH	Not an RDS failure; use the Spark job itself / appropriate runtime command to test

Important Airflow warning

The message 'Multiple rows returned with uselist=False for lazily-loaded attribute DagRun.serialized_dag' was observed during a successful dim_date test. It did not prevent the task from succeeding. Treat it as Airflow metadata cleanup/health work rather than a Spark failure.

13. OPERATIONAL COMMAND REFERENCE

A compact command sheet for future use.

Airflow lifecycle

```
cd /opt/retailhub-airflow
```

```
docker compose up -d
docker compose ps
docker compose logs -f airflow
```

```
docker compose down
docker compose up -d
```

```
# Never use for routine restart:
# docker compose down -v
```

DAG management

```
docker exec retailhub-airflow airflow dags list
docker exec retailhub-airflow airflow dags list | grep retailhub_silver
docker exec retailhub-airflow airflow dags list | grep retailhub_gold
```

```
docker exec retailhub-airflow airflow dags unpause retailhub_silver
docker exec retailhub-airflow airflow dags unpause retailhub_gold
```

```
docker exec retailhub-airflow airflow dags trigger retailhub_silver
docker exec retailhub-airflow airflow dags trigger retailhub_gold
```

```
docker exec retailhub-airflow airflow dags list-runs -d retailhub_silver
docker exec retailhub-airflow airflow dags list-runs -d retailhub_gold
```

Task debugging

```
docker exec retailhub-airflow airflow tasks test    retailhub_gold dim_date 2026-10-05
```

```
docker exec retailhub-airflow airflow tasks states-for-dag-run    retailhub_gold <DAG_RUN_ID>
```

```
docker exec retailhub-airflow airflow tasks states-for-dag-run    retailhub_silver <DAG_RUN_ID>
```

S3 validation

```
aws s3 ls s3://retailhub-bronze-252bda/ --recursive | tail -20
aws s3 ls s3://retailhub-silver-252bda/ --recursive | tail -30
aws s3 ls s3://retailhub-gold-252bda/ --recursive | tail -30
```

```
aws s3 ls s3://retailhub-gold-252bda/ --recursive | grep dim_date
```

Batch validation via SSM

```
aws ssm send-command    --region ap-south-1    --instance-ids i-075044a1c7a02e50f    --document-name "AWS-RunShellScript"    --parameters 'comm
```

```
aws ssm get-command-invocation    --region ap-south-1    --command-id <COMMAND_ID>    --instance-id i-075044a1c7a02e50f
```

14. FINAL VERIFIED STATE AND FUTURE CHECKLIST

What is currently proven

- Airflow EC2 is private and runs Airflow 3.1.0 + PostgreSQL 16 in Docker.
- Airflow DAG files are host-mounted and persistent.
- Airflow logs are host-mounted with correct container permissions.
- Airflow IAM role can call SSM to execute commands on Batch EC2.
- Batch EC2 runs retailhub-spark-batch:4.2.0.
- Silver DAG can remotely execute Spark and write Silver S3 output.
- Gold DAG can remotely execute Spark and write Gold S3 output.
- dim_date was independently tested successfully.
- Gold has 9 tasks and max 3 concurrent tasks.
- Airflow UI is intended to be private and accessed through SSM port forwarding.
- The public website is reached separately through the ALB, not through Airflow.

Next project phases

- Finish/confirm all Gold task runs and S3 Gold outputs.
- Create Glue Data Catalog tables over Gold Parquet.
- Configure Athena and validate analytical SQL.
- Connect Power BI to Athena.
- Run complete Web → Kafka → Streaming → Bronze → Silver → Gold → Athena flow test.
- Add CloudWatch/logging and budget/cost monitoring.
- Tighten IAM and move RDS/Gemini secrets to Secrets Manager for production-quality security.
- Automate the complete architecture with Terraform without changing the proven application flow.

Terraform mapping to preserve

Manual resource	Terraform target
VPC / subnets / routes / NAT	aws_vpc, aws_subnet, aws_route_table, aws_nat_gateway
Security groups	aws_security_group
IAM roles/policies	aws_iam_role, aws_iam_role_policy
S3 buckets	aws_s3_bucket
RDS	aws_db_instance
EC2	aws_instance
ALB/TG/listener	aws_lb, aws_lb_target_group, aws_lb_listener
Airflow Docker setup	user_data / provisioner or image
DAGs	file/template deployment or image

Golden rule for future changes

Keep the proven runtime contract: Airflow schedules → SSM executes → Batch runs the Spark Docker image → Spark reads/writes S3. If moving to Terraform, reproduce this architecture rather than introducing a new orchestration path.

End of reference document. Values marked as placeholders must be filled from the live AWS environment. Passwords and API keys are intentionally omitted from this PDF.